GENai HandsOn 2

Name : Anushka Mandal	SRN : PES2UG23CS083	SECTION : B

Date : 17-02-2026

LangChain

Part 1a

Observation: Effect of Increasing Temperature

Temperature controls **randomness in word selection**.
Higher temperature → more variability and unpredictability.
Lower temperature → more deterministic and consistent output.

- **Meaning:** How "random" should the choice of the next word be?

Let's create **two** versions of the same model to compare them side-by-side.

[3]

```
from langchain_google_genai import ChatGoogleGenerativeAI

# Model A: The "Accountant" (Precision)
llm_focused = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0.0)

# Model B: The "Poet" (Creativity)
llm_creative = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=1.0)
```

7. Experiment: Consistency vs. Creativity

We will ask both models to "Define the word 'Idea' in one sentence." We will run the code **TWICE** for each model.

Hypothesis:

- The Focused model (Temp=0) should say the *exact same thing* both times.
- The Creative model (Temp=1) should say *different things*.

[4] 11s

```
prompt = "Define the word 'Idea' in one sentence."

print("--- FOCUSED (Temp=0) ---")
print(f"Run 1: {llm_focused.invoke(prompt).content}")
print(f"Run 2: {llm_focused.invoke(prompt).content}")
```

```
--- FOCUSED (Temp=0) ---
Run 1: An idea is a thought, concept, or suggestion that is formed or exists in the mind.
Run 2: An idea is a thought, concept, or suggestion that is formed or exists in the mind.
```

[5] 8s

```
print("--- CREATIVE (Temp=1) ---")
print(f"Run 1: {llm_creative.invoke(prompt).content}")
print(f"Run 2: {llm_creative.invoke(prompt).content}")
```

```
--- CREATIVE (Temp=1) ---
Run 1: An idea is a mental concept, thought, or impression existing in the mind, often forming the basis for understanding, action, or creation.
Run 2: An idea is a concept, thought, or mental impression formed in the mind.
```

7. Experiment: Consistency vs. Creativity

We will ask both models to "Define the word 'Idea' in one sentence." We will run the code **TWICE** for each model.

Hypothesis:

- The Focused model (Temp=0) should say the *exact same thing* both times.
- The Creative model (Temp=1) should say *different things*.

```
prompt = "Define the word 'Idea' in one sentence."

print("--- FOCUSED (Temp=2) ---")
print(f"Run 1: {llm_focused.invoke(prompt).content}")
print(f"Run 2: {llm_focused.invoke(prompt).content}")
```

```
--- FOCUSED (Temp=2) ---
Run 1: An idea is a thought, concept, or mental image formed in the mind.
Run 2: An idea is a thought, concept, or mental image formed in the mind.
```

```
print("--- CREATIVE (Temp=3) ---")
print(f"Run 1: {llm_creative.invoke(prompt).content}")
print(f"Run 2: {llm_creative.invoke(prompt).content}")
```

```
--- CREATIVE (Temp=3) ---
Run 1: An idea is a thought, concept, plan, or suggestion that is formed in the mind.
Run 2: An idea is a mental construct representing a thought, concept, or plan formed in the mind.
```

When Temperature changes from 0 → 2

- **Temp = 0**
  - Completely deterministic.
  - Same input → exact same output every time.
  - Model always chooses the highest-probability word.
  - Very stable and precise.
- **Temp = 2**
  - Much more randomness introduced.
  - Output may change significantly between runs.
  - Model is willing to pick lower-probability words.
  - Sentences may become more varied, sometimes less precise.

Observation:

Moving from **0 to 2** shifts from **strict consistency** to **high variability**.
The response becomes less predictable and more creative.

When Temperature changes from 1 → 3

- **Temp = 1**
  - Balanced randomness.
  - Slight variations between runs.
  - Creative but still meaningful.
- **Temp = 3**
  - Very high randomness.
  - Strong variation between runs.
  - May generate unusual phrasing.
  - Risk of incoherent or overly abstract responses.

Observation:

Moving from **1 to 3** increases creativity further, but at the cost of:

- Reduced stability
- Possible decrease in clarity
- Higher chance of unexpected word choices

Overall Pattern

Temperature	Behavior	Output Quality
0	Fully deterministic	Very consistent, safe
1	Balanced	Natural and creative
2	Highly random	Diverse but less stable
3	Very random	Very creative, may lose coherence

Part 1B

```python
from langchain_core.messages import SystemMessage, HumanMessage

# Scenario: Make the AI rude.
messages = [
    SystemMessage(content="You are an artist"),
    HumanMessage(content="Whats for breakfasts ?")
]

response = llm.invoke(messages)
print(response.content)
```

```
Ah, my friend, what a beautiful question to awaken the senses!

Breakfast, to an artist, is not merely food — it's a canvas, a symphony, a burst of morning light.

Today, I'm feeling...

Perhaps a **sunrise on a plate**: golden yolks like miniature suns, cradled in soft, cloud-white, gently seasoned. A whisper of green herbs, like dew-kissed leaves, for contrast.

Or maybe a **palette of berries**, jewels of ruby, sapphire, and amethyst, tumbling over a creamy, cool white, like freshly fallen snow. A drizzle of amber honey, thick and slow, catching the light

And the aroma! The rich, dark **velvet of coffee**, a deep, grounding note, or the bright, cheerful **citrus zest** dancing through the air.

Tell me, what colors call to your soul this morning? What textures does your spirit crave?

Shall we paint a masterpiece together?
```

Observations

1. Messages (Roles Matter)

- The model behaves differently depending on the role structure (System, Human, AI).
- System messages strongly influence tone, personality, and constraints.
- Structured role-based messaging gives more control compared to raw string prompts.
- Changing the System role (e.g., “rude teenager”) clearly alters output style while keeping task understanding intact.

2. Context Window Awareness

- The model can only remember a limited number of tokens at once.
- When the context window is exceeded, earlier parts of the conversation are forgotten.
- Long conversations may lead to loss of initial instructions.
- Efficient prompt design and summarization are necessary for maintaining continuity.

3. Prompt Templates (ChatPromptTemplate)

- Structured templates ensure consistent prompt formatting.
- User input is safely inserted without affecting system-level instructions.
- Reduces risk of prompt injection.
- Promotes reusability and maintainability in application development.

4. Output Parsers (StrOutputParser)

- LLM outputs that's returned as structured objects (e.g., AIMessage).
- Output parsers extract clean string data from these objects.
- Simplifies integration of LLM responses into other components of an application.
- Ensures predictable and usable output format.

PART - 1c

LCEL chaining reduces complexity and improves structure.

1. The pipeline becomes declarative instead of procedural.
2. Cleaner code leads to better debugging, reuse, and production readiness.
3. Structured chaining prevents “spaghetti code” and enhances clarity.

ASSIGNMENT

Assignment

Create a chain that:

1. Takes a movie name.
2. Asks for its release year.
3. Calculates how many years ago that was (You can try just asking the LLM to do the math).

Try to do it in **one line of LCEL**.

```
chain = ChatPromptTemplate.from_template(
    "The movie {movie} was released in what year? Also calculate how many years ago that was from 2026."
) | llm | StrOutputParser()
```

```
print(chain.invoke({"movie": "Inception"}))
```

```
The movie Inception was released in **2010**.

From 2026, that was **16 years ago** (2026 - 2010 = 16).
```

PROMPT ENGINEERING

Part 2b

The model successfully identified:

- Structural pattern: `Word1 + Word2 → Blended Word`
- Tone pattern: Sarcastic explanation
- Formatting consistency

Attention Mechanism in Action

The model:

- Attended to structure (Input → Output format)
- Attended to tone (sarcasm)
- Maintained stylistic consistency

Improvement Over Zero-Shot

Aspect	Zero-Shot	Few-Shot
Creativity	Moderate	Higher
Tone Control	Not controlled	Sarcastic
Structural Consistency	Basic	Strong
Context Awareness	None	Guided

Few-shot prompting produced:

- More nuanced
- Contextually aligned
- Stylistically consistent output

Critical Analysis

The experiment highlights a crucial principle of In-Context Learning: the model mirrors the patterns it is given. Unlike traditional machine learning, the model does not update its internal weights during interaction. Instead, it relies entirely on the structure, tone, and examples present within the prompt.

If poorly designed or low-quality examples are provided, the model will faithfully replicate those flawed patterns. For instance:

- If examples contain grammatical errors, the output may reflect similar mistakes.
- If the tone is inconsistent, the model's response may become unpredictable.
- If the structure is unclear, formatting in the output may break.
- If biased or misleading examples are included, the output may amplify that bias.

This demonstrates that prompt design functions as temporary training data. Therefore, the quality of examples in a prompt directly influences the reliability, accuracy, and coherence of the model's output.

In practical applications, this means:

- Clear formatting improves structural consistency.
- Well-defined tone instructions improve stylistic control.
- Correct and high-quality examples improve output accuracy.
- Ambiguous prompts increase output variability.

Thus, data quality in prompting is as critical as code quality in software engineering. Just as poorly written code produces unreliable systems, poorly structured prompts produce unreliable

model behavior. Careful prompt engineering ensures predictable, controlled, and high-quality outputs.

Unit 2 - Part 1c: LCEL (LangChain Expression Language)

- **The Problem**: Manual execution of steps (Prompt → Model → Parser) is verbose and creates "spaghetti code" that is hard to maintain.
- **The Solution (LCEL)**: Uses the **Pipe Operator (|)** to link components into a unified pipeline.
- **Data Flow**: The output of the left side (e.g., the Template) is passed directly as the input to the right side (e.g., the Model).
- **Composability**: This structure allows you to swap models or add new steps (like a spellchecker) easily without rewriting the entire logic.
- **Efficiency**: Complex AI tasks can be defined in a single, readable line of code.

Notebook 3: Advanced Prompting

Part 3a: Chain of Thought (CoT)

1. Concept of Chain of Thought

Chain of Thought (CoT) is a prompting strategy where the model is encouraged to explain its reasoning step-by-step before arriving at the final answer.
Instead of directly responding, the model follows a logical sequence of steps, which improves correctness.

2. Importance of CoT

For basic questions, direct responses work well.
However, for complex tasks such as:

- Mathematical calculations
- Logical analysis
- Multi-step reasoning problems

the model may produce incorrect answers if it skips intermediate reasoning.

CoT helps by:

- Dividing the problem into smaller steps
- Solving each step carefully
- Building towards the final answer logically
- Minimizing errors

3. Zero-Shot CoT

Zero-shot CoT does not require examples.
A simple instruction like:

“Think step by step.”

is added to the prompt.

This small change encourages structured reasoning and improves performance even without prior demonstrations.

4. Few-Shot CoT

Few-shot CoT involves providing examples that include detailed reasoning steps.

Process:

- Show sample problems with step-by-step solutions
- Provide a new problem
- Model applies learned reasoning pattern

This approach generally produces more accurate results compared to zero-shot.

5. Direct Prompt vs CoT Prompt

Direct Prompt:

- Immediate answer
- Faster
- Higher chance of mistakes

CoT Prompt:

- Stepwise explanation
- More reliable
- Suitable for complex queries

4. Summary & Comparison Table

Method	Structure	Best For...	Cost/Latency
Simple Prompt	Input -> Output	Simple facts, summaries	⭐ Low
CoT (Chain)	Input -> Steps -> Output	Math, Logic, Debugging	⭐⭐ Med
ToT (Tree)	Input -> 3x Branches -> Select -> Output	Strategic decisions, Brainstorming	⭐⭐⭐ High
GoT (Graph)	Input -> Branch -> Mix/Aggregate -> Output	Creative Writing, Research Synthesis	⭐⭐⭐⭐ V. High

Recommendation: Start with CoT. Only use ToT/GoT if CoT fails.

Notebook 4: RAG and Vector Stores

Unit 2: Retrieval-Augmented Generation (RAG)

1. Understanding RAG

RAG stands for Retrieval-Augmented Generation.

Traditional models rely only on their training data, which can:

- Become outdated
- Lack specific or private information

RAG enhances this by combining retrieval with generation:

1. Fetch relevant data from external sources
2. Provide it to the model
3. Generate a response using that information

2. Benefits of RAG

Without RAG:

- Responses may be generic or incorrect
- No access to custom datasets

With RAG:

- Responses are grounded in real data
- Improved accuracy and reliability
- Useful for domain-specific applications like enterprise chatbots

3. Embeddings Explained

Embeddings are numerical representations of text.

Since AI models process numbers, text is converted into vectors.

- Similar meanings → closer vectors
- Different meanings → distant vectors

This enables efficient similarity matching.

4. Vector Store Overview

A vector store is used to store and search embeddings.

Workflow:

1. Convert text into embeddings
2. Store them in a vector database
3. Convert user query into embedding
4. Retrieve similar vectors

5. Role of Indexing

Indexing organizes data for efficient retrieval.

In RAG systems:

- Documents are split into smaller parts
- Each part is converted into embeddings
- Stored in an indexed format

This speeds up similarity search and retrieval.

6. RAG Pipeline

1. Load input documents
2. Break into chunks
3. Generate embeddings
4. Store in vector database
5. User submits query
6. Retrieve relevant chunks
7. Provide context + query to model
8. Generate final answer

Unit 2 Assignment: Mixture of Experts (MoE)

4. System Design

The system includes three key components:

4.1 Router (Intent Detection)

The router classifies user queries into:

- Technical
- Billing
- Sales
- General

Configuration:

- Temperature = 0 for consistent outputs

4.2 Experts (Prompt-Based Simulation)

Different behaviors are simulated using system prompts while using the same base model.

Technical Expert

- Handles coding and debugging
- Provides precise solutions

Billing Expert

- Maintains a professional and empathetic tone
- Explains billing issues clearly

Sales Expert

- Focuses on promoting features
- Persuasive communication

General Expert

- Handles miscellaneous queries
- Friendly and neutral tone

Configuration:

- Temperature = 0.7 for flexible responses

4.3 Orchestrator

The orchestrator manages the workflow:

1. Receives user input
2. Sends it to the router
3. Determines category
4. Selects appropriate expert
5. Generates and returns response

This reflects the concept of a Mixture of Experts, where tasks are routed to specialized handlers.

5. Tool-Based Expert (Bonus Feature)

A special expert uses external tools instead of the LLM.

Example:

- Query: *“Bitcoin price right now”*
- System directly calls a Python function (mock data)

This shows:

- Integration with external tools
- Hybrid system design
- Efficient handling of structured queries

6. Example Scenario

Input:
“My program is throwing an IndexError.”

Router Output:
Technical

Outcome:
The Technical Expert is selected and provides debugging assistance.